
ICPC Algorithm Library

目录	
1 Basic	1
1.1 test	1
1.2 pai	1
1.3 run	1
1.4 缺省源	1
1.5 随机数	1
1.6 Matrix	1
2 Graph	1
2.1 Dijkstra	1
2.2 Tarjan SCC	2
2.3 2-SAT	2
2.4 Segment Tree Graph	2
3 Data Structure	3
3.1 可撤销并查集	3
3.2 单点修改线段树	3
3.3 区间修改线段树	4
3.4 线段树二分	4
3.5 可持久化线段树/动态开点	5
3.6 李超树	5
3.7 线段树合并/分裂	6
3.8 ODT	6
3.9 FHQ Treap	7
3.10 pbds tree	8
3.11 普通莫队	8
3.12 带修莫队	8
3.13 回滚莫队	9
3.14 莫队二次离线	10
4 Tree	10
4.1 占位	10
5 String	10
5.1 KMP	10
5.2 Z Function	10
5.3 String Hash	11
5.4 Manacher	11
5.5 Aho-Corasick	11
5.6 Suffix Array	12
5.7 Suffix Automaton	12
5.8 Palindromic Tree	13
5.9 Lyndon / Minimum Representation	13
6 Geometry	14
6.1 vector	14
6.2 seg_poly	14
7 Number Theory	14
7.1 ModInt	14
7.2 Extended GCD	15
7.3 CRT	15
7.4 Linear Sieve	15
7.5 Miller-Rabin / Pollard-Rho	16
7.6 Primitive Root	17
7.7 exBSGS	17
7.8 Tonelli-Shanks	18

1 Basic

1.1 test

```
export fs="-fsanitize=address,undefined"
ulimit -s 1024000
ulimit -v 1024000
mk() { g++ -o $1 $1.cpp -O2; }
```

1.2 pai

```
pai() {
    mk a; mk bf; mk gen
    while true; do
        ./gen > 1.in
        ./bf < 1.in > 1.ans
        ./a < 1.in > 1.out
        if diff -Zq 1.out 1.ans; then echo ac; else echo wa; break; fi
    done
}
```

1.3 run

```
shopt -s nullglob # 没有匹配到任何文件时, 让它返回空
ulimit -s 1024000
g++ $1.cpp -o $1 -std=c++20 -O2 -fsanitize=address,undefined || exit
for f in "$2"/*.in; do
    x=${f%.in}
    \time -f "$x: %es %MKB" ./$1 < $x.in > $1.out
    diff -Zq $1.out $x.ans
done
```

1.4 缺省源

```
#include <bits/stdc++.h>
using namespace std;

#define ALL(s) s.begin(), s.end()
#define SZ(s) int(s.size())
#define pb push_back

using LL = long long;
using ULL = unsigned long long;
using I = __int128;

void Cas() {
}
int main() {
```

```
cin.tie(0)->sync_with_stdio(0);
int t = 1;
cin >> t;
while (t--) {
    Cas();
}
return 0;
}
```

1.5 随机数

```
ULL seed = chrono::steady_clock::now().time_since_epoch().count();
mt19937_64 rnd(seed);
```

1.6 Matrix

```
template <class T>
struct Mat {
    int n, m;
    vector<T> a;
    Mat(int n = 0, int m = 0, T v = {}) : n(n), m(m), a(n * m, v) {}
    T& operator()(int i, int j) { return a[i * m + j]; }
};

using M = Mat<LL>;
M mul(M a, M b) {
    M c(a.n, b.m);
    for (int i = 0; i < a.n; i++) {
        for (int j = 0; j < b.m; j++) {
            for (int k = 0; k < a.m; k++) {
                c(i, j) = max(c(i, j), a(i, k) + b(k, j));
            }
        }
    }
    return c;
}
```

2 Graph

2.1 Dijkstra

```
struct Node {
    int u;
    LL d;
    bool operator<(const Node& o) const {
        return d > o.d;
    }
};
```

```
priority_queue<Node> q;
vector<LL> D(n, INF);
D[0] = 0;
q.push({0, 0});
while (SZ(q)) {
    auto [u, d] = q.top();
    q.pop();
    if (d != D[u]) continue;
    for (auto [v, w] : G[u]) {
        if (D[v] > d + w) {
            D[v] = d + w;
            q.push({v, D[v]});
        }
    }
}
```

2.2 Tarjan SCC

```
int tim = 0, scc = 0;
vector<int> dfn(n), low(n), bel(n, -1), stk;
auto tarj = [&](auto _, int u) -> void {
    dfn[u] = low[u] = ++tim;
    stk.push_back(u);
    for (int v : G[u]) {
        if (!dfn[v]) {
            _(_, v);
            low[u] = min(low[u], low[v]);
        } else if (bel[v] == -1) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] != low[u]) return;
    while (1) {
        int v = stk.back();
        stk.pop_back();
        bel[v] = scc;
        if (v == u) break;
    }
    scc++;
};
for (int u = 0; u < n; u++) {
    if (!dfn[u]) tarj(tarj, u);
}
```

2.3 2-SAT

- $u_i = 2 \times u + i$
- $u_a \vee v_b$: 加入 $u_a \oplus 1 \rightarrow v_b$ 和 $v_b \oplus 1 \rightarrow u_a$

- $u_a \Rightarrow v_b$: 加入 $u_a \rightarrow v_b$ 和 $v_b \oplus 1 \rightarrow u_a \oplus 1$
- 强制 $u = a$: 加入 $u_a \oplus 1 \rightarrow u_a$
- 禁止 $u = a$ 与 $v = b$ 同时成立: 加入 $u_a \rightarrow v_b \oplus 1$ 和 $v_b \rightarrow u_a \oplus 1$
- $u = v$: 加入 $u_0 \leftrightarrow v_0$ 和 $u_1 \leftrightarrow v_1$
- $u \neq v$: 加入 $u_0 \leftrightarrow v_1$ 和 $u_1 \leftrightarrow v_0$

其中每个 $\leftrightarrow$ 都表示正反两条有向边。

若存在 u 满足 u_0, u_1 在同一强连通分量中, 则无解。

构造: $\text{ans}_u = [\text{bel}_{u_0} > \text{bel}_{u_1}]$ 。

2.4 Segment Tree Graph

- 原图和区间均使用 0-index: 原图节点编号为 $0 \sim n-1$, $\text{mdf}(l, r, \dots)$ 中的 $[l, r]$ 为闭区间。
- ZKW 线段树内部仍使用从 1 开始的堆式编号, 叶子位置为 $S \sim 2S-1$, 原图节点 i 对应叶子 $S+i$ 。
- sn 的值为 $2S-1$, 表示一棵完整 ZKW 线段树的节点数。
- 对于 $1 \leq p \leq \text{sn}$, $\text{id}(p, 0)$ 为 $n+p-1$, 表示 down 树节点; $\text{id}(p, 1)$ 为 $n+\text{sn}+p-1$, 表示 up 树节点。
- tot 初始为 $n+2(2S-1)$, 表示当前节点数, 也是下一个可用节点编号; 初始有效编号为 $0 \sim \text{tot}-1$, $\text{Node}()$ 返回 tot 后将其加一。

```
int S = 1;
while (S < n) S <= 1;
int sn = 2 * S - 1;
int tot = n + 2 * sn;
vector<vector<pair<int, LL>>> G(tot);
auto id = [&](int p, int up) {
    return n + p - 1 + up * sn;
};
auto add = [&](int u, int v, LL w) {
    G[u].push_back({v, w});
};
auto link = [&](int u, int v, LL w, int up) {
    if (up) swap(u, v);
    add(u, v, w);
};
auto Node = [&]() {
    G.emplace_back();
    return tot++;
};
for (int up = 0; up < 2; up++) {
    for (int p = 1; p < S; p++) {
        link(id(p, up), id(p * 2, up), 0, up);
        link(id(p, up), id(p * 2 + 1, up), 0, up);
    }
    for (int i = 0; i < n; i++) {
```

```

    link(id(S + i, up), i, 0, up);
}
}
auto mdf = [&](int l, int r, int x, LL w, int up) {
    for (l += S, r += S + 1; l < r; l >>= 1, r >>= 1) {
        if (l & 1) link(x, id(l++, up), w, up);
        if (r & 1) link(x, id(--r, up), w, up);
    }
};

```

```

// x -> [l, r]
mdf(l, r, x, w, 0);
// [l, r] -> x
mdf(l, r, x, w, 1);
// [l1, r1] -> [l2, r2]
int x = Node();
mdf(l1, r1, x, 0, 1);
mdf(l2, r2, x, w, 0);

```

3 Data Structure

3.1 可撤销并查集

不使用路径压缩，按大小合并；find、merge 单次 $O(\log n)$ 。用 `t=dsu.time()` 保存状态，`dsu.rollback(t)` 撤销此后的所有成功合并。点为 θ -index；`init(n)` 可初始化并用于多测清空。

```

struct DSU {
    vector<int> f, sz;
    vector<array<int, 2>> his;
    DSU(int n = 0) { init(n); }
    void init(int n) {
        f.resize(n), iota(ALL(f), 0);
        sz.assign(n, 1), his.clear();
    }
    int find(int x) {
        while (x != f[x]) x = f[x];
        return x;
    }
    bool same(int x, int y) {
        return find(x) == find(y);
    }
    bool merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        his.push_back({x, y});
        sz[x] += sz[y], f[y] = x;
        return true;
    }
};

```

```

}
int size(int x) {
    return sz[find(x)];
}
int time() {
    return SZ(his);
}
void rollback(int t) {
    while (SZ(his) > t) {
        auto [x, y] = his.back();
        his.pop_back();
        f[y] = y, sz[x] -= sz[y];
    }
}
};

```

3.2 单点修改线段树

- θ -index，左闭右开 $[l, r)$ 。
- 本模板不将 n 补齐到 2 的幂，不能直接读取 `t[1]` 查询全局结果，须使用 `prod(0,  $\hookrightarrow$  n)`。

```

template <class S> struct SGT {
    int n;
    vector<S> t;
    SGT(int n) : n(n), t(2 * n) {}
    SGT(vector<S> a) : SGT(SZ(a)) {
        copy(ALL(a), t.begin() + n);
        for (int p = n - 1; p; p--) t[p] = t[2 * p] + t[2 * p + 1];
    }
    void set(int p, S x) {
        t[p += n] = x;
        while (p >>= 1) t[p] = t[2 * p] + t[2 * p + 1];
    }
    S prod(int l, int r) {
        S x{}, y{};
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) x = x + t[l++];
            if (r & 1) y = t[--r] + y;
        }
        return x + y;
    }
};

```

```

struct S {
    LL mx = -INF;
};
S operator+(S a, S b) {
    return {max(a.mx, b.mx)};
};

```

}

3.3 区间修改线段树

- 0-index, 左闭右开 $[l, r)$ 。
- 按题目补全 V、L::operator=、merge 和 apply, 其中 L{} 为空标记。
- 多组测试每组先用 seg={}; 清空整棵线段树, 再重新 build。

```
#define ls (p << 1)
#define rs (p << 1 | 1)
struct SGT {
    struct V {
    };
    struct L {
        bool operator==(L b) {}
    };
    int n;
    V t[N << 2];
    L lz[N << 2];
    V merge(V a, V b) {}
    void apply(int p, int l, int r, L x) {}
    void up(int p) {
        t[p] = merge(t[ls], t[rs]);
    }
    void down(int p, int l, int r) {
        if (lz[p] == L{}) return;
        int m = (l + r) / 2;
        apply(ls, l, m, lz[p]);
        apply(rs, m, r, lz[p]);
        lz[p] = {};
    }
    void build(const vector<V>& a, int p = 1, int l = 0, int r = -1) {
        if (r < 0) n = r = SZ(a);
        if (r - l == 1) {
            t[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(a, ls, l, m);
        build(a, rs, m, r);
        up(p);
    }
    void mdf(int ql, int qr, L x, int p = 1, int l = 0, int r = -1) {
        if (r < 0) r = n;
        if (ql <= l && r <= qr) return apply(p, l, r, x);
        down(p, l, r);
        int m = (l + r) / 2;
        if (ql < m) mdf(ql, qr, x, ls, l, m);
        if (m < qr) mdf(ql, qr, x, rs, m, r);
    }
};
```

```
up(p);
}
V qry(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (ql <= l && r <= qr) return t[p];
    down(p, l, r);
    int m = (l + r) / 2;
    if (qr <= m) return qry(ql, qr, ls, l, m);
    if (m <= ql) return qry(ql, qr, rs, m, r);
    return merge(qry(ql, qr, ls, l, m), qry(ql, qr, rs, m, r));
}
};
#undef ls
#undef rs
```

```
SGT seg;
seg = {};
seg.build(a);
seg.mdf(l, r, x);
auto ans = seg.qry(l, r);
```

3.4 线段树二分

- 将下列函数直接插入上一节 SGT; 0-index, 左闭右开 $[l, r)$, 无解返回 -1。
- 按题目补全空的 if, 条件表示当前节点区间内没有答案。

```
int find_l(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (r <= ql || qr <= l) return -1;
    if () return -1;
    if (r - l == 1) return l;
    down(p, l, r);
    int m = (l + r) / 2;
    int q = find_l(ql, qr, ls, l, m);
    return q < 0 ? find_l(ql, qr, rs, m, r) : q;
}
int find_r(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (r <= ql || qr <= l) return -1;
    if () return -1;
    if (r - l == 1) return l;
    down(p, l, r);
    int m = (l + r) / 2;
    int q = find_r(ql, qr, rs, m, r);
    return q < 0 ? find_r(ql, qr, ls, l, m) : q;
}
```

3.5 可持久化线段树/动态开点

- 0-index, 左闭右开 $[l, r)$; 按题目补全 v 和 merge , 0 号节点表示空树。
- $\text{mdf}(x, v, p)$ 在版本根 p 上修改位置 x , 返回新根; $\text{qry}(l, r, p)$ 查询版本 p 。
- 多组测试每组先用 $\text{seg}=\{n\}$; 清空节点池并设置长度。

```
#define ls(p) t[p].ls
#define rs(p) t[p].rs
struct SGT {
    struct V {};
    struct Node {
        int ls, rs;
        V v;
    };
    int n;
    vector<Node> t = {{}};
    V merge(V a, V b) {}
    void up(int p) {
        t[p].v = merge(t[ls(p)].v, t[rs(p)].v);
    }
    int mdf(int x, V v, int p = 0, int l = 0, int r = -1) {
        if (r < 0) r = n;
        t.push_back(t[p]);
        p = SZ(t) - 1;
        if (r - l == 1) {
            t[p].v = v;
            return p;
        }
        int m = (l + r) / 2;
        if (x < m) ls(p) = mdf(x, v, ls(p), l, m);
        else rs(p) = mdf(x, v, rs(p), m, r);
        up(p);
        return p;
    }
    V qry(int ql, int qr, int p = 0, int l = 0, int r = -1) {
        if (r < 0) r = n;
        if (ql <= l && r <= qr) return t[p].v;
        int m = (l + r) / 2;
        if (qr <= m) return qry(ql, qr, ls(p), l, m);
        if (m <= ql) return qry(ql, qr, rs(p), m, r);
        return merge(qry(ql, qr, ls(p), l, m), qry(ql, qr, rs(p), m, r));
    }
};
#undef ls
#undef rs
```

```
SGT seg;
seg = {n};
vector<int> rt(q + 1);
rt[i] = seg.mdf(x, v, rt[k]);
```

```
auto ans = seg.qry(l, r, rt[i]);
```

- 若值域很大、只需动态开点而不保留历史版本, 在上面的模板中新增 node , 并将 mdf 替换为下列实现。
- mdf 的根 p 必传, 遇到空节点时新建, 并返回修改后的根; 调用 $\text{rt}=\text{seg.mdf}(x, v, \text{rt})$ 。
- qry 保持不变, 调用时传入 rt ; 0 号节点表示空树。多组测试每组先用 $\text{seg}=\{n\}$; 清空, 再令 $\text{rt}=0$ 。

```
int node() {
    t.push_back({});
    return SZ(t) - 1;
}
int mdf(int x, V v, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p) p = node();
    if (r - l == 1) {
        t[p].v = v;
        return p;
    }
    int m = (l + r) / 2;
    if (x < m) ls(p) = mdf(x, v, ls(p), l, m);
    else rs(p) = mdf(x, v, rs(p), m, r);
    up(p);
    return p;
}
```

3.6 李超树

- 维护整数横坐标 $[0, n)$ 上直线的最小值; 0-index, 左闭右开 $[l, r)$ 。
- $\text{add}(v, p)$ 插入整条直线, $\text{add}(l, r, v, p)$ 插入线段, 均返回根; $\text{merge}(p, q)$ 破坏性合并并返回根, 之后不再使用 q ; $\text{qry}(x, p)$ 查询 x 处的最小值。
- $V\{k, b\}$ 表示 $y = kx + b$; 求最大值时将直线和答案同时取反。多组测试每组先用 $\text{seg} \leftrightarrow \{n\}$; 清空, 再令 $\text{rt}=0$ 。

```
#define ls(p) t[p].ls
#define rs(p) t[p].rs
struct SGT {
    static constexpr LL INF = (LL)4e18;
    struct V {
        LL k = 0, b = INF;
        I get(LL x) { return (I)k * x + b; }
    };
    struct Node {
        int ls, rs;
        V v;
    };
    int n;
    vector<Node> t = {{}};
```

```

int node() {
    t.push_back({});
    return SZ(t) - 1;
}

int add(V v, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p) p = node();
    int m = (l + r) / 2;
    if (v.get(m) < t[p].v.get(m)) swap(v, t[p].v);
    if (r - l == 1) return p;
    if (v.get(l) < t[p].v.get(l)) ls(p) = add(v, ls(p), l, m);
    else if (v.get(r - 1) < t[p].v.get(r - 1)) rs(p) = add(v, rs(p), m, r);
    return p;
}

int add(int ql, int qr, V v, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (ql <= l && r <= qr) return add(v, p, l, r);
    if (!p) p = node();
    int m = (l + r) / 2;
    if (ql < m) ls(p) = add(ql, qr, v, ls(p), l, m);
    if (m < qr) rs(p) = add(ql, qr, v, rs(p), m, r);
    return p;
}

int merge(int p, int q, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p || !q) return p | q;
    p = add(t[q].v, p, l, r);
    if (r - l == 1) return p;
    int m = (l + r) / 2;
    ls(p) = merge(ls(p), ls(q), l, m);
    rs(p) = merge(rs(p), rs(q), m, r);
    return p;
}

LL qry(int x, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p) return INF;
    LL ans = clamp<I>(t[p].v.get(x), -INF, INF);
    if (r - l == 1) return ans;
    int m = (l + r) / 2;
    if (x < m) return min(ans, qry(x, ls(p), l, m));
    return min(ans, qry(x, rs(p), m, r));
}
};
#undef ls
#undef rs

```

3.7 线段树合并/分裂

- `split(l,r,p)` 返回的 `pair` 依次为剩余根和 $[l, r)$ 的根。

```

int merge(int p, int q, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p || !q) return p | q;
    if (r - l == 1) {
        t[p].v = merge(t[p].v, t[q].v);
        return p;
    }
    int m = (l + r) / 2;
    ls(p) = merge(ls(p), ls(q), l, m);
    rs(p) = merge(rs(p), rs(q), m, r);
    up(p);
    return p;
}

pair<int, int> split(int ql, int qr, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p || qr <= l || r <= ql) return {p, 0};
    if (ql <= l && r <= qr) return {0, p};
    int m = (l + r) / 2;
    int al = ls(p), ar = rs(p), bl = 0, br = 0;
    if (ql < m) tie(al, bl) = split(ql, qr, al, l, m);
    if (m < qr) tie(ar, br) = split(ql, qr, ar, m, r);
    ls(p) = al, rs(p) = ar;
    if (al || ar) up(p);
    else p = 0;
    if (!bl && !br) return {p, 0};
    int q = node();
    ls(q) = bl, rs(q) = br;
    up(q);
    return {p, q};
}

```

3.8 ODT

```

struct ODT {
    struct Node {
        int l, r, c;
        bool operator<(Node x) const {
            return l < x.l;
        }
    };
    set<Node> s;
    using IT = set<Node>::iterator;
    IT split(int x) {
        auto it = s.lower_bound({x});
        if (it != s.end() && it->l == x) return it;
        auto p = prev(it);
        if (p->r == x) return it;
        Node a = *p;

```

```

    s.erase(p);
    s.insert({a.l, x, a.c});
    return s.insert({x, a.r, a.c}).first;
}
IT merge(IT it) {
    Node a = *it;
    if (it != s.begin() && prev(it)->c == a.c) {
        a.l = prev(it)->l;
        s.erase(prev(it));
    }
    auto q = next(it);
    if (q != s.end() && q->c == a.c) {
        a.r = q->r;
        s.erase(q);
    }
    s.erase(it);
    return s.insert(a).first;
}
void mdf(int l, int r, int c) {
    auto R = split(r), L = split(l);
    for (auto it = L; it != R; ++it) {
    }
    s.erase(L, R);
    merge(s.insert({l, r, c}).first);
}
};

```

```

ODT odt;
odt = {}; // 每组测试清空
odt.s.insert({0, n, c}); // 必须先完整覆盖 [0, n)
odt.mdf(l, r, x);

```

3.9 FHQ Treap

```

#define ls(p) t[p].ch[0]
#define rs(p) t[p].ch[1]
struct FHQTreap {
    struct V {
    };
    struct Node {
        int ch[2]{}; sz = 0;
        ULL pri = 0;
        V v;
        bool rev = false;
    };
    vector<Node> t = {};
    int rt = 0;
    int size(int p) { return t[p].sz; }
    int size() { return size(rt); }
}

```

```

void up(int p) {
    t[p].sz = size(ls(p)) + size(rs(p)) + 1;
}
void apply_rev(int p) {
    if (p) swap(ls(p), rs(p)), t[p].rev ^= 1;
}
void down(int p) {
    if (!t[p].rev) return;
    apply_rev(ls(p)), apply_rev(rs(p));
    t[p].rev = false;
}
int node(V v) {
    t.push_back({{ }, 1, rnd(), v});
    return SZ(t) - 1;
}
void split(int p, int k, int& x, int& y) {
    if (!p) return x = y = 0, void();
    down(p);
    if (k <= size(ls(p))) y = p, split(ls(p), k, x, ls(y));
    else x = p, split(rs(p), k - size(ls(p)) - 1, rs(x), y);
    up(p);
}
int merge(int p, int q) {
    if (!p || !q) return p | q;
    if (t[p].pri < t[q].pri) {
        down(p), rs(p) = merge(rs(p), q), up(p);
        return p;
    }
    down(q), ls(q) = merge(p, ls(q)), up(q);
    return q;
}
int insert(int p, int k, int q) {
    if (!p) return q;
    if (t[q].pri < t[p].pri) {
        split(p, k, ls(q), rs(q)), up(q);
        return q;
    }
    down(p);
    if (k <= size(ls(p))) ls(p) = insert(ls(p), k, q);
    else rs(p) = insert(rs(p), k - size(ls(p)) - 1, q);
    up(p);
    return p;
}
void insert(int k, V v) {
    rt = insert(rt, k, node(v));
}
int erase(int p, int k) {
    down(p);
    int s = size(ls(p));
    if (k == s) return merge(ls(p), rs(p));
}

```

```

    if (k < s) ls(p) = erase(ls(p), k);
    else rs(p) = erase(rs(p), k - s - 1);
    up(p);
    return p;
}
void erase(int k) {
    rt = erase(rt, k);
}
void reverse(int l, int r) {
    int x, y, z;
    split(rt, r, x, z), split(x, l, x, y);
    apply_rev(y), rt = merge(merge(x, y), z);
}
int kth(int p, int k) {
    down(p);
    int s = size(ls(p));
    if (k == s) return p;
    if (k < s) return kth(ls(p), k);
    return kth(rs(p), k - s - 1);
}
void clear() {
    t.assign(1, {}), rt = 0;
}
};
#undef ls
#undef rs

```

```

FHQTreap tr;
for (int i = 0; i < n; i++) tr.insert(i, {}); // 初始化
tr.clear(); // 多测清空

```

3.10 pbds tree

- `tr.order_of_key(x)` 返回严格小于 `x` 的元素个数。
- `tr.find_by_order(k)` 返回 `0-index` 第 `k` 小元素的迭代器；若 `k>=tr.size()`，返回 `tr.end()`。

```

#include <bits/extc++.h>
using namespace __gnu_pbds;
template <class T>
using Tree = tree<T, null_type, std::less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

```

3.11 普通莫队

离线处理区间询问。若加入、删除一个位置的复杂度为 A ，取块长 $B \approx \sqrt{n}$ ，总复杂度为 $O((n+q)\sqrt{n}A)$ 。查询使用 `0-index`，左闭右开 $[l, r)$ ；分别在 `add`、`del`、`answer` 中维护答案。多测使用局部变量或执行 `mo={};`。

```

struct Mo {
    struct Q {
        int l, r, id;
    };
    vector<Q> q;
    void add(int p) {
    }
    void del(int p) {
    }
    void answer(int id) {
    }
    void ask(int l, int r) {
        q.push_back({l, r, SZ(q)});
    }
    void solve(int n) {
        int B = max(1, int(sqrt(n)));
        sort(ALL(q), [&](Q a, Q b) {
            int x = a.l / B, y = b.l / B;
            if (x != y) return x < y;
            return x & 1 ? a.r > b.r : a.r < b.r;
        });
        int l = 0, r = 0;
        for (auto [ql, qr, id] : q) {
            while (ql < l) add(--l);
            while (r < qr) add(r++);
            while (l < ql) del(l++);
            while (qr < r) del(--r);
            answer(id);
        }
    }
};

```

3.12 带修莫队

将修改次数作为第三维。按输入顺序调用 `modify` 和 `ask`，每个询问自动记录此前的修改次数；先令 `a` 为初始序列，并补全 `V`、`add`、`del`、`answer`。查询使用 `0-index`，左闭右开 $[l, r)$ 。三维规模同阶时取 $B \approx n^{2/3}$ ，复杂度为 $O(n^{5/3}A)$ ；多测使用局部变量或执行 `mo={};`。

```

struct ModifyMo {
    struct V {
    };
    struct Q {
        int l, r, t, id;
    };
    struct C {
        int p;
        V v;
    };
};

```

```

vector<V> a;
vector<Q> q;
vector<C> c;
void add(int p) {
}
void del(int p) {
}
void answer(int id) {
}
void modify(int p, V v) {
    c.push_back({p, v});
}
void ask(int l, int r) {
    q.push_back({l, r, SZ(c), SZ(q)});
}
void apply(int k, int l, int r) {
    auto& [p, v] = c[k];
    if (l <= p && p < r) del(p);
    swap(a[p], v);
    if (l <= p && p < r) add(p);
}
void solve() {
    int B = max(1, int(pow(SZ(a), 2.0 / 3)));
    sort(ALL(q), [&](Q x, Q y) {
        int bx = x.l / B, by = y.l / B;
        if (bx != by) return bx < by;
        bx = x.r / B, by = y.r / B;
        if (bx != by) return bx < by;
        return x.t < y.t;
    });
    int l = 0, r = 0, t = 0;
    for (auto [ql, qr, qt, id] : q) {
        while (t < qt) apply(t++, l, r);
        while (qt < t) apply(--t, l, r);
        while (ql < l) add(--l);
        while (r < qr) add(r++);
        while (l < ql) del(l++);
        while (qr < r) del(--r);
        answer(id);
    }
}
};

```

3.13 回滚莫队

适用于好加难删。按左端点分块，块内右端点单调增加。左端点临时加入，同块询问暴力处理，回答后均撤销。查询使用 θ -index，左闭右开 $[l, r)$ 。补全 add、snapshot、rollback、answer，其中每次 add 的全部影响都必须可撤销。复杂度为 $O((n+q)\sqrt{n}A)$ ；

好删难加时反向处理。

```

struct RollbackMo {
    struct Q {
        int l, r, id;
    };
    vector<Q> q;
    void add(int p) {
    }
    int snapshot() {
        return 0;
    }
    void rollback(int t) {
    }
    void answer(int id) {
    }
    void ask(int l, int r) {
        q.push_back({l, r, SZ(q)});
    }
    void solve(int n) {
        int B = max(1, int(sqrt(n)));
        sort(ALL(q), [&](Q a, Q b) {
            int x = a.l / B, y = b.l / B;
            if (x != y) return x < y;
            return a.r < b.r;
        });
        for (int i = 0; i < SZ(q); i) {
            int b = q[i].l / B, j = i;
            while (j < SZ(q) && q[j].l / B == b) j++;
            int R = min(n, (b + 1) * B), r = R;
            int base = snapshot();
            for (int k = i; k < j; k++) {
                auto [ql, qr, id] = q[k];
                if (qr <= R) {
                    int t = snapshot();
                    for (int p = ql; p < qr; p++) add(p);
                    answer(id), rollback(t);
                    continue;
                }
                while (r < qr) add(r++);
                int t = snapshot();
                for (int p = R - 1; p >= ql; p--) add(p);
                answer(id), rollback(t);
            }
            rollback(base), i = j;
        }
    }
};

```

3.14 莫队二次离线

当单步转移为 $O(\log n)$ ，但贡献关于区间可以作前缀差分，将莫队的连续指针移动再次离线，批量计算每次询问相对上一次询问的变化量。move_l、move_r 分别把一整段端点移动压成 $O(1)$ 个 E，不要逐点枚举；calc 扫描所有事件并累加到 d[id]。最后按莫队顺序对变化量求前缀和得到 ans。E 的字段和三处函数均随题目定义；逆序对中配合值域分块可做到 $O(n\sqrt{q} + n\sqrt{n})$ 时间、 $O(n + q)$ 空间。

```
struct OfflineMo {
    struct Q {
        int l, r, id;
    };
    struct E {
    };
    vector<Q> q;
    vector<E> e;
    vector<LL> d, ans;
    void move_l(int l, int ql, int r, int id) {
    }
    void move_r(int r, int qr, int l, int id) {
    }
    void calc() {
    }
    void ask(int l, int r) {
        q.push_back({l, r, SZ(q)});
    }
    void solve(int n) {
        int B = max(1, int(sqrt(n)));
        sort(ALL(q), [&](Q a, Q b) {
            int x = a.l / B, y = b.l / B;
            if (x != y) return x < y;
            return x & 1 ? a.r > b.r : a.r < b.r;
        });
        e.clear(), d.assign(SZ(q), 0), ans.resize(SZ(q));
        vector<int> ord;
        int l = 0, r = 0;
        for (auto [ql, qr, id] : q) {
            if (l != ql) move_l(l, ql, r, id);
            if (r != qr) move_r(r, qr, ql, id);
            l = ql, r = qr, ord.push_back(id);
        }
        calc();
        LL cur = 0;
        for (int id : ord) cur += d[id], ans[id] = cur;
    }
};
```

4 Tree

4.1 占位

TODO：待补充树相关模板。

5 String

5.1 KMP

- kmp(s)[i] 表示 $s[0, i)$ 的最长相等真前后缀长度，返回数组长度为 $n + 1$ 。
- 模板只依赖 size() 和元素判等，可直接用于字符串或一般序列。

```
template <class T> vector<int> kmp(const T& s) {
    int n = s.size();
    vector<int> f(n + 1);
    for (int i = 1, j = 0; i < n; i++) {
        while (j && s[i] != s[j]) j = f[j];
        if (s[i] == s[j]) j++;
        f[i + 1] = j;
    }
    return f;
}
```

5.2 Z Function

- ZFunc(s)[i] 表示 s 与后缀 $s[i, n)$ 的最长公共前缀长度，并令 $z[0]=n$ 。
- exKMP(s, t, z) 返回每个后缀 $s[i, n)$ 与模式串 t 的最长公共前缀长度，其中 $z = \text{ZFunc}(s \rightarrow t)$ 。

```
template <class T> vector<int> ZFunc(const T& s) {
    int n = s.size(), l = 0, r = 0;
    vector<int> z(n);
    if (!n) return z;
    z[0] = n;
    for (int i = 1; i < n; i++) {
        if (i <= r && z[i - l] < r - i + 1) z[i] = z[i - l];
        else {
            z[i] = max(0, r - i + 1);
            while (i + z[i] < n && s[i + z[i]] == s[z[i]]) z[i]++;
            if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
        }
    }
    return z;
}

template <class T>
vector<int> exKMP(const T& s, const T& t, const vector<int>& z) {
    int n = s.size(), m = t.size(), l = 0, r = 0;
    vector<int> p(n);
    if (!n) return p;
    while (p[0] < n && p[0] < m && s[p[0]] == t[p[0]]) p[0]++;
}
```

```

for (int i = 1; i < n; i++) {
    if (i <= r && z[i - l] < r - i + 1) p[i] = z[i - l];
    else {
        p[i] = max(0, r - i + 1);
        while (i + p[i] < n && p[i] < m && s[i + p[i]] == t[p[i]]) p[i]++;
        if (i + p[i] - 1 > r) l = i, r = i + p[i] - 1;
    }
}
return p;
}

```

5.3 String Hash

- 双模 Hash，所有位置均为 0-index，get(l,r) 返回半开区间 $s[l, r)$ 的归一化 Hash。
- 两段字符串长度相同且 Hash 相等时可认为相等；Hash 算法始终存在极小碰撞概率。

```

struct StringHash {
    using H = pair<LL, LL>;
    static constexpr LL P1 = 2034452107, P2 = 2013074419;
    static constexpr LL B1 = 137, B2 = 149;
    int n;
    vector<LL> pw1, pw2, h1, h2;
    StringHash(const string& s) : n(s.size()),
        pw1(n + 1, 1), pw2(n + 1, 1), h1(n + 1), h2(n + 1) {
        for (int i = 0; i < n; i++) {
            int x = (unsigned char)s[i] + 1;
            pw1[i + 1] = pw1[i] * B1 % P1;
            pw2[i + 1] = pw2[i] * B2 % P2;
            h1[i + 1] = (h1[i] * B1 + x) % P1;
            h2[i + 1] = (h2[i] * B2 + x) % P2;
        }
    }
    H get(int l, int r) {
        assert(0 <= l && l <= r && r <= n);
        LL x = (h1[r] - h1[l] * pw1[r - l] % P1 + P1) % P1;
        LL y = (h2[r] - h2[l] * pw2[r - l] % P2 + P2) % P2;
        return {x, y};
    }
};

```

5.4 Manacher

- 算法在变换串 $\#s[0]\#s[1]\#\dots\#$ 上运行， $p[i]-1$ 是以变换串位置 i 为中心的最长回文子串在原串中的长度。
- 对应原串半开区间为 $[(i-p[i]+1)/2, (i+p[i]-1)/2)$ 。

```

vector<int> Manacher(const string& s) {
    string t = "#";

```

```

    for (char c : s) t += c, t += '#';
    int n = t.size();
    vector<int> p(n);
    for (int i = 0, l = 0, r = 0; i < n; i++) {
        p[i] = i < r ? min(p[2 * l - i], r - i) : 0;
        while (i + p[i] < n && i - p[i] >= 0
            && t[i + p[i]] == t[i - p[i]]) p[i]++;
        if (i + p[i] > r) l = i, r = i + p[i];
    }
    return p;
}

```

5.5 Aho-Corasick

- 字符集固定为小写字母；先用 add(s) 插入全部模式串并保存返回的终止节点，再调用一次 build()。
- count(s)[p] 是节点 p 对应模式串在文本 s 中的出现次数，重复模式串会共享终止节点。

```

struct ACAM {
    struct Node {
        array<int, 26> ch{};
        int fail = 0;
    };
    vector<Node> t = {{}};
    vector<int> ord;
    ACAM(int n = 0) { t.reserve(n + 1); }
    int add(const string& s) {
        int p = 0;
        for (char x : s) {
            int c = x - 'a';
            assert(0 <= c && c < 26);
            if (!t[p].ch[c]) {
                t[p].ch[c] = t.size();
                t.push_back({});
            }
            p = t[p].ch[c];
        }
        return p;
    }
    void build() {
        queue<int> q;
        ord.clear();
        for (int c = 0; c < 26; c++) {
            if (t[0].ch[c]) q.push(t[0].ch[c]);
        }
        while (SZ(q)) {
            int u = q.front();
            q.pop();

```

```

ord.pb(u);
for (int c = 0; c < 26; c++) {
    int v = t[u].ch[c];
    if (v) {
        t[v].fail = t[t[u].fail].ch[c];
        q.push(v);
    } else {
        t[u].ch[c] = t[t[u].fail].ch[c];
    }
}
}
vector<LL> count(const string& s) {
    vector<LL> cnt(t.size());
    int p = 0;
    for (char x : s) {
        int c = x - 'a';
        assert(0 <= c && c < 26);
        p = t[p].ch[c];
        cnt[p]++;
    }
    for (int i = SZ(ord) - 1; i >= 0; i--) {
        int u = ord[i];
        cnt[t[u].fail] += cnt[u];
    }
    return cnt;
}
};

```

5.6 Suffix Array

- $sa[i]$ 是排名第 i 的后缀起点, $rk[i]$ 是后缀 $s[i, n)$ 的排名, 均为 0-index。
- $lc[i]$ 等于 $\text{lcp}(s[sa[i], n), s[sa[i+1], n))$, 数组长度为 $n - 1$ 。

```

struct SuffixArray {
    int n;
    vector<int> sa, rk, lc;
    SuffixArray(const string& s) : n(s.size()), sa(n), rk(n), lc(max(0, n - 1)) {
        if (!n) return;
        iota(ALL(sa), 0);
        sort(ALL(sa), [&](int a, int b) { return s[a] < s[b]; });
        rk[sa[0]] = 0;
        for (int i = 1; i < n; i++) {
            rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
        }
        vector<int> tmp, cnt(n);
        tmp.reserve(n);
        for (int w = 1; rk[sa[n - 1]] < n - 1; w *= 2) {
            tmp.clear();

```

```

            for (int i = max(0, n - w); i < n; i++) tmp.pb(i);
            for (int i = 0; i < n; i++) if (sa[i] >= w) tmp.pb(sa[i] - w);
            fill(ALL(cnt), 0);
            for (int i = 0; i < n; i++) cnt[rk[i]]++;
            partial_sum(ALL(cnt), cnt.begin());
            for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[tmp[i]]]] = tmp[i];
            swap(rk, tmp);
            rk[sa[0]] = 0;
            for (int i = 1; i < n; i++) {
                int a = sa[i - 1], b = sa[i];
                int x = a + w < n ? tmp[a + w] : -1;
                int y = b + w < n ? tmp[b + w] : -1;
                rk[b] = rk[a] + (tmp[a] != tmp[b] || x != y);
            }
        }
        for (int i = 0, k = 0; i < n; i++) {
            int j = rk[i];
            if (!j) {
                k = 0;
                continue;
            }
            if (k) k--;
            int p = sa[j - 1];
            while (i + k < n && p + k < n && s[i + k] == s[p + k]) k++;
            lc[j - 1] = k;
        }
    }
};

```

5.7 Suffix Automaton

- 根节点编号为 1; 普通字符串从 $p=1$ 开始, 依次令 $p=\text{sam.ins}(p, c - 'a')$ 。
- 最多产生 $2n$ 个节点; fa 是后缀链接, len 是状态所表示子串的最大长度。

```

template <int C = 26>
struct SAM {
    vector<array<int, C>> ch;
    vector<int> fa, len;
    SAM(int n = 0) : ch(2), fa(2), len(2) {
        ch.reserve(2 * n + 2);
        fa.reserve(2 * n + 2);
        len.reserve(2 * n + 2);
    }
    int node() {
        ch.push_back({});
        fa.pb(0);
        len.pb(0);
        return SZ(ch) - 1;
    }
};

```

```

int ins(int p, int c) {
    assert(0 <= c && c < C);
    int cur = node();
    len[cur] = len[p] + 1;
    for (; p && !ch[p][c]; p = fa[p]) ch[p][c] = cur;
    if (!p) fa[cur] = 1;
    else {
        int q = ch[p][c];
        if (len[q] == len[p] + 1) fa[cur] = q;
        else {
            int cl = node();
            len[cl] = len[p] + 1;
            ch[cl] = ch[q];
            fa[cl] = fa[q];
            fa[q] = fa[cur] = cl;
            for (; p && ch[p][c] == q; p = fa[p]) ch[p][c] = cl;
        }
    }
    return cur;
}
};

```

5.8 Palindromic Tree

- 字符须先映射到 $[0, C)$ ；调用 `init(n)` 后逐个 `add(c)`，节点 0、1 分别是长度 0、-1 的根。
- `size()` 是本质不同回文串数，`suf_cnt()` 是当前回文后缀数，`count()` 是已经插入串的回文子串总数。
- 全部插入后调用一次 `calc()`，即可由 `t[p].cnt` 得到每个回文串的出现次数。

```

template <int C = 26>
struct PAM {
    struct Node {
        array<int, C> ch{};
        int fail = 0, len = 0, cnt = 0, num = 0;
    };
    vector<Node> t;
    vector<int> s;
    int las;
    LL tot;
    void init(int n) {
        t.assign(2, {});
        t.reserve(n + 2);
        t[0].fail = t[1].fail = 1;
        t[1].len = -1;
        s.assign(1, -1);
        s.reserve(n + 1);
        las = 0;
        tot = 0;
    }
};

```

```

}
int get_fail(int p, int c) {
    int n = SZ(s) - 1;
    while (s[n - t[p].len - 1] != c) p = t[p].fail;
    return p;
}
int add(int c) {
    assert(0 <= c && c < C);
    s.pb(c);
    int p = get_fail(las, c);
    if (!t[p].ch[c]) {
        int q = get_fail(t[p].fail, c);
        Node x;
        x.len = t[p].len + 2;
        x.fail = t[q].ch[c];
        x.num = t[x.fail].num + 1;
        t[p].ch[c] = SZ(t);
        t.pb(x);
    }
    las = t[p].ch[c];
    t[las].cnt++;
    tot += t[las].num;
    return las;
}
int size() { return SZ(t) - 2; }
int suf_cnt() { return t[las].num; }
LL count() { return tot; }
void calc() {
    for (int p = SZ(t) - 1; p >= 2; p--) t[t[p].fail].cnt += t[p].cnt;
}
};

```

5.9 Lyndon / Minimum Representation

- `Duval(s)` 返回 Lyndon 分解的 0-index 半开区间，按顺序拼接后恰为原序列。
- `min_rep(s)` 返回循环同构序列中字典序最小表示的起点；空序列返回 0。

```

template <class T>
vector<pair<int, int>> Duval(const T& s) {
    int n = s.size(), i = 0;
    vector<pair<int, int>> a;
    while (i < n) {
        int j = i, k = i + 1;
        while (k < n && s[j] <= s[k]) {
            if (s[j] < s[k]) j = i;
            else j++;
            k++;
        }
        while (i <= j) {

```

```

    int r = i + k - j;
    a.push_back({i, r});
    i = r;
}
}
return a;
}
template <class T> int min_rep(const T& s) {
    int n = s.size(), i = 0, j = 1, k = 0;
    if (!n) return 0;
    while (i < n && j < n && k < n) {
        if (s[(i + k) % n] == s[(j + k) % n]) k++;
        else {
            if (s[(i + k) % n] > s[(j + k) % n]) i += k + 1;
            else j += k + 1;
            if (i == j) j++;
            k = 0;
        }
    }
    return min(i, j);
}
}

```

6 Geometry

6.1 vector

```

using db = long double;
struct p2 {
    db x, y;
    db len() { return hypot(x, y); }
    p2 operator+(p2 b) { return {x + b.x, y + b.y}; }
    p2 operator-(p2 b) { return {x - b.x, y - b.y}; }
    p2 operator*(db k) { return {x * k, y * k}; }
    p2 operator/(db k) { return {x / k, y / k}; }
    db operator*(p2 b) { return x * b.x + y * b.y; }
    db operator%(p2 b) { return x * b.y - y * b.x; }
};
using ps = vector<p2>;

```

6.2 seg_poly

```

// pa 到 pb 的叉积，逆时针转为正
db side(p2 p, p2 a, p2 b) { return (a - p) % (b - p); }
db tol(p2 p, p2 a, p2 b) { return abs(side(p, a, b)) / (b - a).len(); }

```

7 Number Theory

7.1 ModInt

- 修改 P 可更换模数；除法和 inv() 要求 P 为质数且除数非零。
- qp(a,b) 要求指数 $b \geq 0$ ，也可用于其他定义了乘法的类型。

```

constexpr int P = 998244353;
template <class T> T qp(T a, LL b) {
    assert(b >= 0);
    T r{1};
    for (; b >>= 1, a *= a) if (b & 1) r *= a;
    return r;
}
struct Mint {
    int v;
    Mint(LL x = 0) : v(int((x % P + P) % P)) {}
    int val() const { return v; }
    Mint operator-() const { return Mint(-v); }
    Mint& operator+=(const Mint& b) {
        if ((v += b.v) >= P) v -= P;
        return *this;
    }
    Mint& operator-=(const Mint& b) {
        if ((v -= b.v) < 0) v += P;
        return *this;
    }
    Mint& operator*=(const Mint& b) {
        v = int(LL(v) * b.v % P);
        return *this;
    }
    Mint& operator/=(const Mint& b) { return *this *= b.inv(); }
    friend Mint operator+(Mint a, const Mint& b) { return a += b; }
    friend Mint operator-(Mint a, const Mint& b) { return a -= b; }
    friend Mint operator*(Mint a, const Mint& b) { return a *= b; }
    friend Mint operator/(Mint a, const Mint& b) { return a /= b; }
    Mint inv() const {
        assert(v);
        return qp(*this, P - 2);
    }
    friend istream& operator>>(istream& is, Mint& a) {
        LL x;
        is >> x;
        return a = Mint(x), is;
    }
    friend ostream& operator<<(ostream& os, const Mint& a) {
        return os << a.v;
    }
};
using mint = Mint;

```

7.2 Extended GCD

- `exgcd(a,b,x,y)` 要求 $a, b \geq 0$ 且不同时为零, 返回 $g = \gcd(a, b)$, 并满足 $ax + by = g$ 。
- `linear_congruence(a,b,m)` 求 $ax \equiv b \pmod{m}$ 。无解返回 `nullopt`; 否则返回 $\{r, p\}$, 全部解为 $x \equiv r \pmod{p}$ 。

```
LL exgcd(LL a, LL b, LL& x, LL& y) {
    assert(a >= 0 && b >= 0 && (a || b));
    if (!b) return x = 1, y = 0, a;
    LL g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}

optional<pair<LL, LL>> linear_congruence(LL a, LL b, LL m) {
    assert(m > 0);
    a %= m;
    b %= m;
    if (a < 0) a += m;
    if (b < 0) b += m;
    LL x, y, g = exgcd(a, m, x, y);
    if (b % g) return nullopt;
    LL p = m / g;
    I r = I(x) * (b / g) % p;
    if (r < 0) r += p;
    return pair<LL, LL>{LL(r), p};
}
```

7.3 CRT

- 合并任意个方程 $x \equiv r_i \pmod{m_i}$, 模数不要求互质, 但必须为正数。
- 返回 $\{r, m\}$ 表示全部解为 $x \equiv r \pmod{m}$; 无解返回 $\{0, 0\}$ 。要求最终模数最小公倍数不超过 `LL`。

```
LL safe_mod(LL x, LL m) {
    x %= m;
    return x < 0 ? x + m : x;
}

pair<LL, LL> inv_gcd(LL a, LL b) {
    a = safe_mod(a, b);
    if (!a) return {b, 0};
    LL s = b, t = a;
    I x = 0, y = 1;
    while (t) {
        LL q = s / t;
        s -= q * t;
        swap(s, t);
        x -= I(q) * y;
        swap(x, y);
    }
}
```

```
LL m = b / s;
x %= m;
if (x < 0) x += m;
return {s, LL(x)};
}

pair<LL, LL> crt(const vector<LL>& r, const vector<LL>& m) {
    assert(r.size() == m.size());
    LL r0 = 0, m0 = 1;
    for (int i = 0; i < SZ(r); i++) {
        assert(m[i] > 0);
        LL r1 = safe_mod(r[i], m[i]), m1 = m[i];
        if (m0 < m1) swap(r0, r1), swap(m0, m1);
        if (m0 % m1 == 0) {
            if (r0 % m1 != r1) return {0, 0};
            continue;
        }
        auto [g, im] = inv_gcd(m0, m1);
        LL u = m1 / g;
        if ((r1 - r0) % g) return {0, 0};
        LL x = LL(I((r1 - r0) / g) * im % u);
        I nm = I(m0) * u;
        assert(nm <= numeric_limits<LL>::max());
        I nr = r0 + I(x) * m0;
        m0 = LL(nm);
        r0 = LL(nr % m0);
        if (r0 < 0) r0 += m0;
    }
    return {r0, m0};
}
```

7.4 Linear Sieve

- $O(n)$ 求出不超过 n 的全部质数、最小质因子、欧拉函数 ϕ 和莫比乌斯函数 μ 。
- `factor(x)` 适用于 $1 \leq x \leq n$, 返回 {质因子, 次数}。

```
struct Sieve {
    int n;
    vector<int> pr, minp, phi, mu;
    Sieve(int n_) : n(n_), minp(n + 1), phi(n + 1), mu(n + 1) {
        assert(n >= 1);
        phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; i++) {
            if (!minp[i]) {
                pr.push_back(i);
                minp[i] = i;
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : pr) {

```

```

        if (LL(i) * p > n) break;
        minp[i * p] = p;
        if (i % p == 0) {
            phi[i * p] = phi[i] * p;
            break;
        }
        phi[i * p] = phi[i] * (p - 1);
        mu[i * p] = -mu[i];
    }
}
vector<pair<int, int>> factor(int x) const {
    assert(1 <= x && x <= n);
    vector<pair<int, int>> f;
    while (x > 1) {
        int p = minp[x], c = 0;
        do x /= p, c++;
        while (x > 1 && minp[x] == p);
        f.push_back({p, c});
    }
    return f;
}
};

```

7.5 Miller-Rabin / Pollard-Rho

- `prime(n)` 对整个 ULL 范围使用确定性 Miller-Rabin; `factor(n)` 返回带重数且升序排列的质因子。
- 乘法使用 `__uint128_t`, 要求编译器支持 GNU C++; `factor(1)` 返回空数组。

```

namespace Factor {
using U128 = __uint128_t;
ULL mul_mod(ULL a, ULL b, ULL m) {
    return ULL(U128(a) * b % m);
}
ULL add_mod(ULL a, ULL b, ULL m) {
    return ULL((U128(a) + b) % m);
}
ULL pow_mod(ULL a, ULL b, ULL m) {
    ULL r = 1 % m;
    for (; b >= 1, a = mul_mod(a, a, m))
        if (b & 1) r = mul_mod(r, a, m);
    return r;
}
bool prime(ULL n) {
    if (n < 2) return false;
    for (ULL p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
        19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }
}

```

```

}
int s = __builtin_ctzll(n - 1);
ULL d = (n - 1) >> s;
for (ULL a : {2ULL, 325ULL, 9375ULL, 28178ULL,
    450775ULL, 9780504ULL, 1795265022ULL}) {
    if (a % n == 0) continue;
    ULL x = pow_mod(a % n, d, n);
    if (x == 1 || x == n - 1) continue;
    bool ok = false;
    for (int r = 1; r < s; r++) {
        x = mul_mod(x, x, n);
        if (x == n - 1) {
            ok = true;
            break;
        }
    }
    if (!ok) return false;
}
return true;
}
ULL diff(ULL a, ULL b) { return a > b ? a - b : b - a; }
ULL rho(ULL n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    static mt19937_64 rng(chrono::steady_clock::now()
        .time_since_epoch().count());
    while (true) {
        ULL y = rng() % (n - 2) + 2;
        ULL c = rng() % (n - 1) + 1;
        auto f = [&](ULL x) { return add_mod(mul_mod(x, x, n), c, n); };
        ULL g = 1, x = 0, z = 0;
        for (ULL r = 1; g == 1; r <= 1) {
            x = y;
            for (ULL i = 0; i < r; i++) y = f(y);
            for (ULL k = 0; k < r && g == 1; k += 128) {
                z = y;
                ULL q = 1;
                for (ULL i = 0; i < min<ULL>(128, r - k); i++) {
                    y = f(y);
                    q = mul_mod(q, diff(x, y), n);
                }
                g = gcd(q, n);
            }
        }
    }
    if (g == n) {
        do z = f(z), g = gcd(diff(x, z), n);
        while (g == 1);
    }
    if (g != n) return g;
}
}

```

```

}
void factor(ULL n, vector<ULL>& f) {
    if (n == 1) return;
    if (prime(n)) return f.push_back(n);
    ULL d = rho(n);
    factor(d, f);
    factor(n / d, f);
}
vector<ULL> factor(ULL n) {
    assert(n > 0);
    vector<ULL> f;
    factor(n, f);
    sort(f.begin(), f.end());
    return f;
}
vector<pair<ULL, int>> factor_count(ULL n) {
    vector<ULL> a = factor(n);
    vector<pair<ULL, int>> f;
    for (ULL p : a) {
        if (f.empty() || f.back().first != p) f.push_back({p, 1});
        else f.back().second++;
    }
    return f;
}
}
}

```

7.6 Primitive Root

- 求质数 p 的最小正原根，依赖前一个 Miller-Rabin / Pollard-Rho 条目。
- 这里只处理质数模数；`primitive_root(2)=1`。

```

ULL primitive_root(ULL p) {
    assert(Factor::prime(p));
    if (p == 2) return 1;
    vector<ULL> f = Factor::factor(p - 1);
    f.erase(unique(f.begin(), f.end()), f.end());
    for (ULL g = 2; g < p; g++) {
        bool ok = true;
        for (ULL q : f) {
            if (Factor::pow_mod(g, (p - 1) / q, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return g;
    }
    return 0;
}
}

```

7.7 exBSGS

- 求最小的 $x \geq 0$ 使 $a^x \equiv b \pmod{m}$ ，模数不要求为质数；无解返回 -1。
- 时间与空间复杂度均为 $O(\sqrt{m})$ ，使用哈希表保存 baby steps。

```

namespace DiscreteLog {
LL norm(LL x, LL m) {
    x %= m;
    return x < 0 ? x + m : x;
}
LL mul_mod(LL a, LL b, LL m) { return LL(I(a) * b % m); }
LL inv_mod(LL a, LL m) {
    LL b = m;
    I x = 1, y = 0;
    while (b) {
        LL q = a / b;
        a -= q * b;
        swap(a, b);
        x -= I(q) * y;
        swap(x, y);
    }
    assert(a == 1);
    x %= m;
    if (x < 0) x += m;
    return LL(x);
}
LL bsgs(LL a, LL b, LL m) {
    assert(m > 1 && gcd(a, m) == 1);
    LL n = LL(sqrtl((long double)m)) + 1;
    unordered_map<LL, LL> baby;
    baby.reserve(size_t(2 * n + 1));
    LL cur = 1 % m;
    for (LL j = 0; j < n; j++) {
        baby.emplace(cur, j);
        cur = mul_mod(cur, a, m);
    }
    LL step = inv_mod(cur, m);
    cur = b;
    for (LL i = 0; i <= n; i++) {
        auto it = baby.find(cur);
        if (it != baby.end()) {
            I x = I(i) * n + it->second;
            if (x <= numeric_limits<LL>::max()) return LL(x);
            return -1;
        }
        cur = mul_mod(cur, step, m);
    }
    return -1;
}
LL exbsgs(LL a, LL b, LL m) {

```

```

assert(m > 0);
if (m == 1) return 0;
a = norm(a, m);
b = norm(b, m);
if (b == 1) return 0;
LL k = 1, add = 0;
while (true) {
    LL g = gcd(a, m);
    if (g == 1) break;
    if (b % g) return -1;
    b /= g;
    m /= g;
    k = mul_mod(k, a / g, m);
    add++;
    if (k == b) return add;
}
b = mul_mod(b, inv_mod(k, m), m);
LL x = bsgs(a, b, m);
return x < 0 ? -1 : x + add;
}
}
using DiscreteLog::exbsgs;

```

7.8 Tonelli-Shanks

- `mod_sqrt(a,p)` 要求 p 为质数，返回较小的平方根；二次非剩余返回-1。
- 另一根为 $p-r$ 。复杂度为 $O(\log^2 p)$ ，乘法使用 `__int128` 防止 LL 溢出。

```

LL sqrt_mul(LL a, LL b, LL p) { return LL(I(a) * b % p); }
LL sqrt_pow(LL a, LL b, LL p) {
    LL r = 1 % p;
    for (; b >= 1, a = sqrt_mul(a, a, p))
        if (b & 1) r = sqrt_mul(r, a, p);
    return r;
}
LL mod_sqrt(LL a, LL p) {
    assert(p >= 2);
    a %= p;
    if (a < 0) a += p;
    if (!a || p == 2) return a;
    if (sqrt_pow(a, (p - 1) / 2, p) != 1) return -1;
    if (p % 4 == 3) {
        LL x = sqrt_pow(a, (p + 1) / 4, p);
        return min(x, p - x);
    }
    LL q = p - 1; int s = 0;
    while (!(q & 1)) q >>= 1, s++;
    LL z = 2; while (sqrt_pow(z, (p - 1) / 2, p) != p - 1) z++;
    LL c = sqrt_pow(z, q, p), x = sqrt_pow(a, (q + 1) / 2, p);

```

```

LL t = sqrt_pow(a, q, p); int m = s;
while (t != 1) {
    int i = 0; LL y = t;
    while (y != 1 && i < m) y = sqrt_mul(y, y, p), i++;
    assert(i < m);
    LL b = c;
    for (int j = 0; j < m - i - 1; j++) b = sqrt_mul(b, b, p);
    x = sqrt_mul(x, b, p);
    c = sqrt_mul(b, b, p);
    t = sqrt_mul(t, c, p);
    m = i;
}
return min(x, p - x);
}

```